

Way Back Home - Level 0：確認身分

來源：<https://codelabs.developers.google.com/way-back-home-level-0/instructions?hl=zh-tw>

步驟數：8

使用 Gemini 的多輪圖像生成功能，製作獨一無二的太空探險家顯示圖片

目錄

1. [求救信號](#)
2. [準備環境](#)
3. [連線至任務控管中心](#)
4. [自訂多層檢視](#)
5. [建構虛擬化身產生器](#)
6. [產生及註冊](#)
7. [結論](#)
8. [額外步驟：以相片為基礎的虛擬化身 \(選填\)](#)

步驟 1 · 約 0 分鐘

1. 求救信號



你的逃生艙劃破未知世界的的大氣層，每個面板都會閃爍警示燈。導航系統壞掉了，通訊服務中斷。太空艙的緊急通訊協定啟動後，AI 助理開始運作：

「偵測到倖存者，緊急降落成功。*Initiating rescue beacon protocol...*」

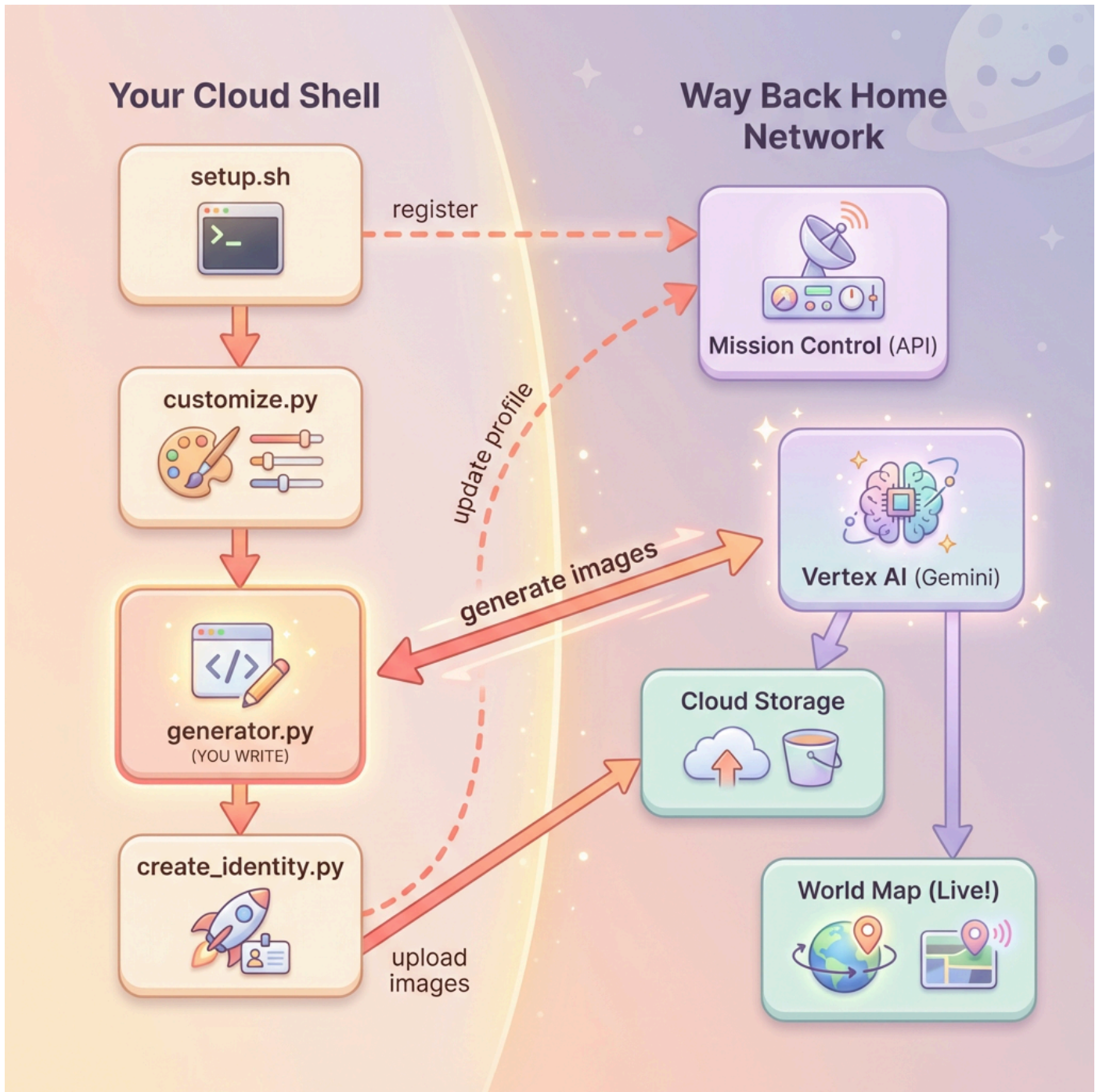
「警告：必須完成身分驗證。銀河救援網路無法找到未註冊的探險家。請確認您的身分，以啟用信標。」

你從觀景窗往外看，外星地景延伸至地平線，有奇異的岩層結構、不熟悉的植被和顏色怪異的天空。在某處，你任務中的其他倖存者散落在這個星球的表面。

但首先，您必須存在於系統中。

建構項目

在這個關卡中，您將使用 Google 的 Gemini 圖像生成模型 (又稱 [Nano Banana](#) 🍌) 進行多輪圖像生成，打造專屬的太空探險家身分。您的虛擬人偶會顯示在全球救援地圖上，讓工作坊活動中的所有其他倖存者都能看到。



課程內容

概念	說明
多輪圖像生成	如何透過對話工作階段，在多張生成圖像中維持角色一致性
圖片提示工程	設計有效的提示，在特定限制下生成風格一致的輸出內容
Gemini Image API (Nano Banana)	透過 Python SDK 使用 Gemini 的原生圖像生成功能
對話工作階段	運用對話脈絡進行反覆修正，並確保角色一致性

完成本級別後，您將：

- ✔ 使用文字轉圖像 AI 生成探索者的肖像
- ✔ 使用多輪對話建立一致的地圖圖示
- ✔ 向救援網路註冊身分
- ✔ 與其他探索者一起顯示在即時世界地圖上

讓我們在地圖上顯示你的位置！📍

步驟 2 · 約 0 分鐘

2. 準備環境

info

從做中學 免付費

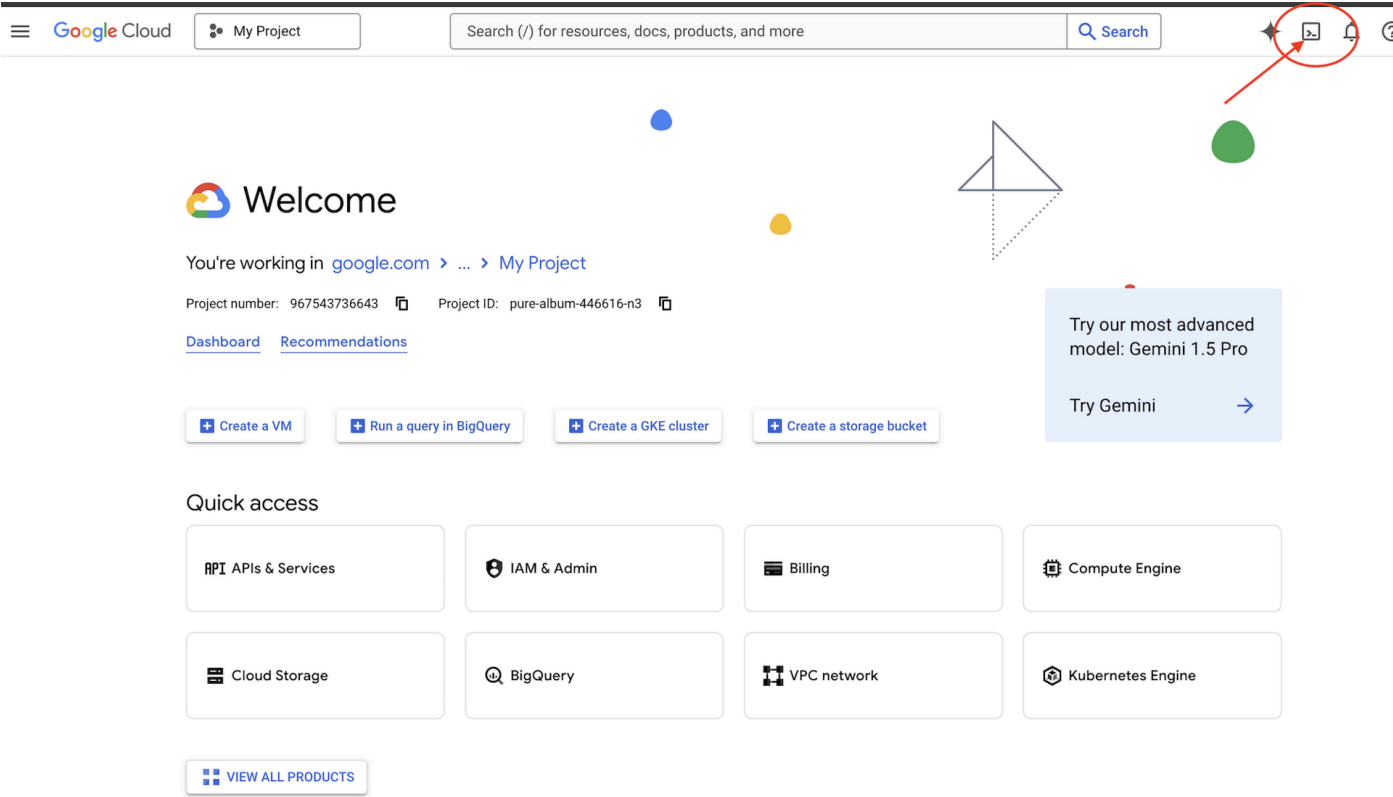
立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

存取 Cloud Shell

首先，我們要開啟 Cloud Shell。這是以瀏覽器為基礎的終端機，已預先安裝 Google Cloud SDK 和其他必要工具。

點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」（右上角導覽列中的終端機圖示）。



開啟 Cloud Shell 後，請確認您已通過驗證：

```
# Check that you are logged in
gcloud auth list
```

您的帳戶應該會顯示為 (ACTIVE) 。

複製存放區

複製 Way Back Home 存放區，然後前往專案：

```
git clone https://github.com/gca-americas/way-back-home.git
cd way-back-home
```

安裝依附元件

前往 Level 0 並安裝必要的 Python 套件：

```
cd level_0
uv sync
```

主要依附元件如下：

套件	目的
<code>google-genai</code>	用於生成圖片的 Gemini API 用戶端
<code>requests</code>	用於對 Mission Control 進行 API 呼叫的 HTTP 用戶端
<code>Pillow</code>	圖片處理和檔案處理

驗證設定

快速驗證，確保所有設定皆正確無誤：

```
uv run python ../scripts/verify_setup.py
```

如下所示：

```
✓ Authenticated as: your-email@google.com
✓ Python environment ready (uv)
✓ Ready to proceed!
```

疑難排解

如果驗證失敗：

- 未通過驗證：執行 `gcloud auth login`
- 缺少依附元件：從 `level_0` 目錄重新執行 `uv sync`

您剛完成的項目

- ✓ 開啟 Cloud Shell
- ✓ 向 Google Cloud 進行驗證
- ✓ 複製程式碼研究室存放區

✓ 使用 `uv`

安裝 Python 依附元件 ✓ 驗證設定

後續步驟：連線至 Mission Control，設定指令碼會自動設定 Google 雲端專案和 API。

步驟 3 · 約 0 分鐘

3. 連線至任務控管中心

執行設定指令碼

設定指令碼會將你連線至 Way Back Home 救援網路，並保留你的探索者身分。從專案根目錄執行：

```
cd $HOME/way-back-home
chmod +x scripts/setup.sh
./scripts/setup.sh
```

系統會提示您輸入兩項資訊。

輸入活動代碼

活動代碼可識別你參加的講習課程活動。

如果你在研討會現場：輸入 QR code、投影片或研討會講師提供的活動代碼。

```
🚀 Welcome to Way Back Home!

Enter event code (from QR/slide): devfest-nyc-26
Validating event...
✓ Connected to: DevFest NYC 2026
```

如果是自學：輸入 `sandbox` 即可加入公開學習環境。

```
🚀 Welcome to Way Back Home!

Enter event code (from QR/slide): sandbox
Validating event...
✓ Connected to: Way Back Home Sandbox
```

沙箱環境

`sandbox` 活動是永久公開的環境，適合自學者。您將與世界各地的其他學員分享地圖，瞭解自己並非孤軍奮戰！沙箱每週都會重設，因此不必擔心會變得雜亂。

找不到活動？

請仔細檢查活動代碼。這項屬性會區分大小寫，通常會遵循 `city-event-year` 等模式。如果你是在研討會，請詢問老師。如果是獨自學習，請使用 `sandbox`。

事件驗證的運作方式

輸入活動代碼後，設定指令碼會呼叫 Mission Control API，確認活動是否存在：

```
GET https://api.waybackhome.dev/events/devfest-nyc-26
```

API 會檢查有效研討會活動的資料庫。如果有效，則會傳回事件名稱和設定。確保你連線至實際進行中的研討會。

選擇探索者名稱

為探索器選擇專屬名稱，世界地圖和排行榜會顯示此個人資料。

```
Choose your explorer name: AstroAyo
✓ Username available!
```

如果所選名稱已由同一活動的其他出席者使用：

```
Choose your explorer name: SpaceExplorer
⚠ That name is taken. Try another.
Choose your explorer name: SpaceExplorer42
✓ Username available!
```

安裝完成

完成後，你會看到確認訊息：

```
Initializing your explorer profile...

✓ Environment configured!
  Explorer ID: alb2c3d4
  Starting coordinates: (47, 23)

Next: cd level_0 && python customize.py
```

發生了什麼事？

設定指令碼對 Mission Control 發出多項 API 呼叫：

1. 已驗證活動代碼：確認研討會活動存在且有效
2. 檢查使用者名稱是否可用：確保活動中沒有其他人使用該名稱
3. 保留身分：建立具有專屬 ID 的參與者名額
4. 指派的起始座標：將你放在地球表面的隨機位置
5. 已儲存設定 - 將所有詳細資料寫入專案根目錄中的 `config.json`

前往第 0 層

現在前往 Level 0 目錄 (您已從第 2 模組安裝依附元件)：

```
cd level_0
```

環境提醒

如果您開啟了新終端機，請從 `level_0` 目錄執行 `uv sync`，確保環境已準備就緒。您也可以使用 `uv run python` 執行下列所有 `python` 指令，自動使用正確的環境。

檢查設定

查看儲存的内容 (`config.json` 位於專案根目錄中)：

```
cat ../config.json
```

```
{
  "event_code": "devfest-nyc-26",
  "event_name": "DevFest NYC 2026",
  "username": "AstroAyo",
  "participant_id": "a1b2c3d4",
  "starting_x": 47,
  "starting_y": 23,
  "api_base": "https://api.waybackhome.dev",
  "project_id": "your-project-id"
}
```

後續指令碼會使用這個設定檔，向救援網路驗證您的身分。

您剛完成的項目

- ✓ 已連線至 Mission Control API
- ✓ 已預留專屬探索者名稱
- ✓ 已收到參與者 ID 和起始座標
- ✓ 已儲存設定，可進行後續步驟

下一步：自訂多層檢視的外觀。

步驟 4 · 約 0 分鐘

4. 自訂多層檢視

執行自訂指令碼

生成虛擬化身前，請先選擇探索者的外觀：

```
uv run python customize.py
```

選取西裝顏色

選擇探險家的太空裝顏色：

```
🎨 Let's create your explorer identity!
```

```
Select suit color:
```

1. Deep Blue
2. Crimson Red
3. Forest Green
4. Royal Purple
5. Solar Gold
6. Silver

```
Choice [1-6, default=6]: 1
```

```
✓ Deep Blue selected
```

描述你的 Explorer (選填)

你可以簡短描述探索者的外觀，或按 Enter 隨機指派：

```
Brief description of your explorer (or Enter for random):
```

```
Example: 'short dark hair, glasses, friendly smile'
```

```
> short dark hair, glasses, determined expression
```

```
✓ Preferences saved!
```

如果未輸入任何內容就按下 Enter 鍵，系統會隨機產生特徵：

```
>
```

```
✓ Random traits: confident expression, short styled hair
```

撰寫優質說明的訣竅

說明應著重於可見的身體特徵：

- **良好**：「捲髮、紅髮、雀斑、溫暖的笑容」
- **良好**：「銀髮盤起、眼神銳利、飽經風霜的臉」
- **避免**：冗長的身世背景或非視覺特徵

AI 會在生成肖像時解讀你的描述，因此清楚的視覺細節效果最佳。

已更新設定

系統現在會儲存你的偏好設定：

✓ Preferences saved!

Next: Open `generator.py` and follow the codelab instructions

驗證更新後的設定：

```
cat ../config.json
```

您會看到新增的偏好設定：

```
{
  "event_code": "devfest-nyc-26",
  "event_name": "DevFest NYC 2026",
  "username": "AstroAyo",
  "participant_id": "a1b2c3d4",
  "starting_x": 47,
  "starting_y": 23,
  "api_base": "https://api.waybackhome.dev",
  "project_id": "your-project-id",
  "suit_color": "deep blue with silver accents",
  "appearance": "short dark hair, glasses, determined expression"
}
```

您剛完成的項目

- ✓ 選取套裝顏色
- ✓ 定義探索者的外觀
- ✓ 設定完成，可生成圖片

下一步：主要活動，也就是撰寫圖片生成程式碼！

步驟 5 · 約 0 分鐘

5. 建構虛擬化身產生器

這是核心學習單元。您將編寫 Python 程式碼，使用 Gemini 的多輪圖像生成功能 (Nano Banana) 生成專屬的探險家虛擬人偶。

開啟產生器檔案

在 Cloud Shell 編輯器中開啟虛擬人偶產生器：

```
cloudshell edit generator.py
```

或者，在 Cloud Shell 中點選「開啟編輯器」，然後前往 `level_0` 資料夾中的 `generator.py`。

瞭解檔案結構

這個檔案包含範例程式碼和三個預留位置區段，您將在這些區段中新增實作項目：

```

"""
Level 0: Avatar Generator

This module generates your unique space explorer avatar using
multi-turn image generation with Gemini (Nano Banana) for
character consistency across portrait and icon.
"""

from google import genai
from google.genai import types
from PIL import Image
import json
import os
import io

# Load configuration from setup (config.json is in project root)
CONFIG_PATH = "../config.json"

with open(CONFIG_PATH) as f:
    config = json.load(f)

USERNAME = config["username"]
SUIT_COLOR = config["suit_color"]
APPEARANCE = config["appearance"]

# Initialize the Gemini client for Vertex AI
client = genai.Client(
    vertexai=True,
    project=os.environ.get("GOOGLE_CLOUD_PROJECT", config.get("project_id")),
    location="us-central1"
)

def generate_explorer_avatar() -> dict:
    """
    Generate portrait and icon using multi-turn chat for consistency.

    The key technique here is using a CHAT SESSION rather than independent
    API calls. This allows Gemini to "remember" the character it created
    in the first turn, ensuring the icon matches the portrait.

    Returns:
        dict with portrait_path and icon_path
    """

    # MODULE_5_STEP_1_CREATE_CHAT_SESSION
    # TODO: Create a chat session for multi-turn generation
    chat = None # Replace this line

    # MODULE_5_STEP_2_GENERATE_PORTRAIT
    # TODO: Generate the explorer portrait
    portrait_image = None # Replace this section

```

```

# MODULE_5_STEP_3_GENERATE_ICON
# TODO: Generate a consistent map icon
icon_image = None # Replace this section

return {
    "portrait_path": "outputs/portrait.png",
    "icon_path": "outputs/icon.png"
}

if __name__ == "__main__":
    # Create outputs directory if it doesn't exist
    os.makedirs("outputs", exist_ok=True)

    print(f"Generating avatar for {USERNAME}...")
    result = generate_explorer_avatar()
    print(f"✅ Avatar created!")
    print(f"    Portrait: {result['portrait_path']}")
    print(f"    Icon: {result['icon_path']}")

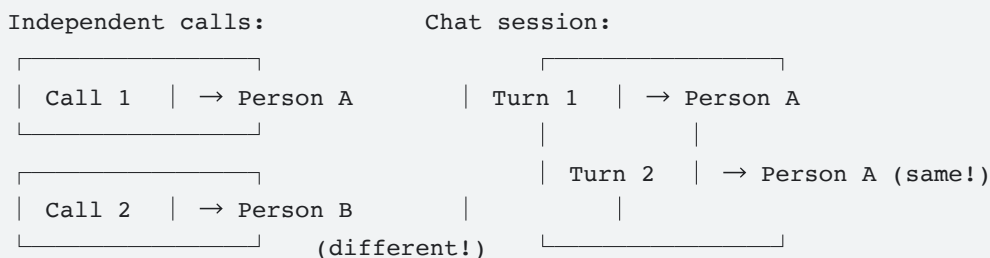
```

您會將三個 `TODO` 區段替換為自己的實作項目。

重要概念：多輪對話可確保角色一致性

如要在多張圖片中生成一致的角色，訣竅是使用**即時通訊工作階段**。在對話中生成圖片時，Gemini 會記住生成的內容。在同一個對話中，後續訊息可以提及「這個角色」或「同一個人」，Gemini 會維持視覺一致性。

這與進行個別獨立的 API 呼叫有本質上的差異，因為後者每次都會產生不同的字元，沒有共用內容。



步驟 1：建立即時通訊工作階段

找出預留位置 `MODULE_5_STEP_1_CREATE_CHAT_SESSION`，並將 `chat = None # Replace this line` 行替換為：

```
# MODULE_5_STEP_1_CREATE_CHAT_SESSION
# Create a chat session to maintain character consistency across generations.
# The chat session preserves context between turns, so Gemini "remembers"
# what it generated and can create consistent variations.
chat = client.chats.create(
    model="gemini-2.5-flash-image", # Nano Banana - Gemini with image generation
    config=types.GenerateContentConfig(
        response_modalities=["TEXT", "IMAGE"]
    )
)
```

瞭解程式碼

- `client.chats.create()`：建立新的對話工作階段，在多輪對話中保留脈絡
- `model="gemini-2.5-flash-image"`：使用 Gemini 2.5 Flash 模型，並採用原生圖像生成功能 (Nano Banana)
- `response_modalities=["TEXT", "IMAGE"]`：告訴 Gemini 我們希望回覆中包含文字說明和生成的圖片。這是生成圖片的必要資訊。

步驟 2：生成肖像

找出 `MODULE_5_STEP_2_GENERATE_PORTRAIT`，然後將 `portrait_image = None # Replace this section` 替換為下列程式碼：


```
# MODULE_5_STEP_2_GENERATE_PORTRAIT
# First turn: Generate the explorer portrait.
# This establishes the character that will be referenced in subsequent turns.
portrait_prompt = f"""Create a stylized space explorer portrait.
```

```
Character appearance: {APPEARANCE}
```

```
Name on suit patch: "{USERNAME}"
```

```
Suit color: {SUIT_COLOR}
```

```
CRITICAL STYLE REQUIREMENTS:
```

- Digital illustration style, clean lines, vibrant saturated colors
- Futuristic but weathered space suit with visible mission patches
- Background: Pure solid white (#FFFFFF) - absolutely no gradients, patterns, or elements
- Frame: Head and shoulders only, 3/4 view facing slightly left
- Lighting: Soft diffused studio lighting, no harsh shadows
- Expression: Determined but approachable
- Art style: Modern animated movie character portrait (similar to Pixar or Dreamworks style)

```
The white background is essential - the avatar will be composited onto a map."""
```

```
print("🌈 Generating your portrait...")
```

```
portrait_response = chat.send_message(portrait_prompt)
```

```
# Extract the image from the response.
```

```
# Gemini returns a response with multiple "parts" - we need to find the image part.
```

```
portrait_image = None
```

```
for part in portrait_response.candidates[0].content.parts:
```

```
    if part.inline_data is not None:
```

```
        # Found the image! Convert from bytes to PIL Image and save.
```

```
        image_bytes = part.inline_data.data
```

```
        portrait_image = Image.open(io.BytesIO(image_bytes))
```

```
        portrait_image.save("outputs/portrait.png")
```

```
        break
```

```
if portrait_image is None:
```

```
    raise Exception("Failed to generate portrait - no image in response")
```

```
print("✓ Portrait generated!")
```

提示工程訣竅

請注意提示的結構：

1. **主體定義**：「製作風格獨特的太空探險家肖像」可讓 Gemini 確切瞭解我們的需求
2. **變數插入**：我們會插入使用者的選擇 ({APPEARANCE}、{USERNAME}、{SUIT_COLOR})
3. **風格規定**：特定藝術方向 (「數位插畫」、「皮克斯或夢工廠風格」)
4. **技術限制**：白色背景、特定構圖、照明

純白色背景限制對於「回家路線」體驗至關重要，可讓虛擬人偶乾淨地合成到地圖上，不會發生色彩衝突或出現邊緣偽影。

瞭解回應剖析

Gemini 的回覆包含多個「部分」：

- **文字部分**：模型提供的註解或說明
- **圖片部分**：以二進位資料形式生成的圖片 (`inline_data`)

我們會逐一檢查各個部分，尋找包含原始圖片位元組的 `inline_data`。接著，我們會將這些位元組轉換為 PIL 圖片，以便儲存。

步驟 3：生成地圖圖示

找出 `MODULE_5_STEP_3_GENERATE_ICON`，並將 `icon_image = None # Replace this section` 替換為：

```
# MODULE_5_STEP_3_GENERATE_ICON
# Second turn: Generate a consistent icon for the map.
# Because we're in the same chat session, Gemini remembers the character
# from the portrait and will maintain visual consistency.
icon_prompt = """Now create a circular map icon of this SAME character.
```

CRITICAL REQUIREMENTS:

- SAME person, SAME face, SAME expression, SAME suit – maintain perfect consistency with the portrait
- Tighter crop: just the head and very top of shoulders
- Background: Pure solid white (#FFFFFF)
- Optimized for small display sizes (will be used as a 64px map marker)
- Keep the exact same art style, colors, and lighting as the portrait
- Square 1:1 aspect ratio

This icon must be immediately recognizable as the same character from the portrait."""

```
print("🗺️ Creating map icon...")
icon_response = chat.send_message(icon_prompt)

# Extract the icon image from the response
icon_image = None
for part in icon_response.candidates[0].content.parts:
    if part.inline_data is not None:
        image_bytes = part.inline_data.data
        icon_image = Image.open(io.BytesIO(image_bytes))
        icon_image.save("outputs/icon.png")
        break

if icon_image is None:
    raise Exception("Failed to generate icon - no image in response")

print("✓ Icon generated!")
```

重要概念：透過內容保持一致性

請注意，圖示提示會說明「這個相同角色」，並強調「相同的人、相同的臉、相同的表情、相同的西裝」。

由於我們在同一個即時通訊工作階段中，Gemini 知道我們指的是哪個角色，也就是在上一輪對話中建立的角色。模型會自動維護這個脈絡。

如果我們啟動新的即時通訊工作階段或發出獨立的 API 呼叫，Gemini 會生成完全不同外觀的人像，因為系統不會記憶先前的生成結果。

您完成的程式碼

新增所有三個區段後，`generate_explorer_avatar()` 函式應如下所示：

```

def generate_explorer_avatar() -> dict:
    """
    Generate portrait and icon using multi-turn chat for consistency.

    The key technique here is using a CHAT SESSION rather than independent
    API calls. This allows Gemini to "remember" the character it created
    in the first turn, ensuring the icon matches the portrait.

    Returns:
        dict with portrait_path and icon_path
    """

    # MODULE_5_STEP_1_CREATE_CHAT_SESSION
    # Create a chat session to maintain character consistency across generations.
    # The chat session preserves context between turns, so Gemini "remembers"
    # what it generated and can create consistent variations.
    chat = client.chats.create(
        model="gemini-2.5-flash-image", # Nano Banana - Gemini with image generation
        config=types.GenerateContentConfig(
            response_modalities=["TEXT", "IMAGE"]
        )
    )

    # MODULE_5_STEP_2_GENERATE_PORTRAIT
    # First turn: Generate the explorer portrait.
    # This establishes the character that will be referenced in subsequent turns.
    portrait_prompt = f"""Create a stylized space explorer portrait.

    Character appearance: {APPEARANCE}
    Name on suit patch: "{USERNAME}"
    Suit color: {SUIT_COLOR}

    CRITICAL STYLE REQUIREMENTS:
    - Digital illustration style, clean lines, vibrant saturated colors
    - Futuristic but weathered space suit with visible mission patches
    - Background: Pure solid white (#FFFFFF) - absolutely no gradients, patterns, or elements
    - Frame: Head and shoulders only, 3/4 view facing slightly left
    - Lighting: Soft diffused studio lighting, no harsh shadows
    - Expression: Determined but approachable
    - Art style: Modern animated movie character portrait (similar to Pixar or Dreamworks style)

    The white background is essential - the avatar will be composited onto a map."""

    print("🌈 Generating your portrait...")
    portrait_response = chat.send_message(portrait_prompt)

    # Extract the image from the response.
    # Gemini returns a response with multiple "parts" - we need to find the image part.
    portrait_image = None
    for part in portrait_response.candidates[0].content.parts:
        if part.inline_data is not None:
            # Found the image! Convert from bytes to PIL Image and save.

```

```

        image_bytes = part.inline_data.data
        portrait_image = Image.open(io.BytesIO(image_bytes))
        portrait_image.save("outputs/portrait.png")
        break

if portrait_image is None:
    raise Exception("Failed to generate portrait - no image in response")

print("✓ Portrait generated!")

# MODULE_5_STEP_3_GENERATE_ICON
# Second turn: Generate a consistent icon for the map.
# Because we're in the same chat session, Gemini remembers the character
# from the portrait and will maintain visual consistency.
icon_prompt = """Now create a circular map icon of this SAME character.
```

CRITICAL REQUIREMENTS:

- SAME person, SAME face, SAME expression, SAME suit – maintain perfect consistency with the portrait
- Tighter crop: just the head and very top of shoulders
- Background: Pure solid white (#FFFFFF)
- Optimized for small display sizes (will be used as a 64px map marker)
- Keep the exact same art style, colors, and lighting as the portrait
- Square 1:1 aspect ratio

This icon must be immediately recognizable as the same character from the portrait."""

```

print("🗺️ Creating map icon...")
icon_response = chat.send_message(icon_prompt)

# Extract the icon image from the response
icon_image = None
for part in icon_response.candidates[0].content.parts:
    if part.inline_data is not None:
        image_bytes = part.inline_data.data
        icon_image = Image.open(io.BytesIO(image_bytes))
        icon_image.save("outputs/icon.png")
        break

if icon_image is None:
    raise Exception("Failed to generate icon - no image in response")

print("✓ Icon generated!")

return {
    "portrait_path": "outputs/portrait.png",
    "icon_path": "outputs/icon.png"
}
```

儲存檔案

請務必省下 `generator.py` :

- **Cloud Shell 編輯器**：Ctrl+S (Windows/Linux) 或 Cmd+S (Mac)
- **vim**：按下 Escape 鍵，然後輸入 `:wq` 並按下 Enter 鍵

您剛建構的內容

- ✓ 建立即時通訊工作階段，進行多輪圖像生成
- ✓ 撰寫詳細的肖像生成提示，並設定樣式限制
- ✓ 運用對話內容生成一致的地圖圖示
- ✓ 瞭解如何剖析 Gemini 回覆中的圖像資料

已掌握的重要概念：

概念	您學到的內容
對話工作階段	<code>client.chats.create()</code> 會在多輪對話中保留脈絡
回覆方式	<code>["TEXT", "IMAGE"]</code> 可在回覆中生成圖像
提示結構	主題 → 變數 → 樣式 → 技術限制
角色一致性	同一場即時通訊工作階段 = 圖片中的角色相同
剖析回應	從回應部分的 <code>inline_data</code> 中擷取圖片

後續步驟：執行程式碼，在地圖上查看自己的位置！

步驟 6 · 約 0 分鐘

6. 產生及註冊

設定專案 ID

確認專案 ID 可做為環境變數：

```
export GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project)
```

執行身分建立工具

現在請執行主要指令碼，協調產生、上傳及註冊作業：

```
uv run python create_identity.py
```

系統會即時顯示生成進度：

注意：你的顯示圖片可能需要最多 5 秒才會顯示在線上地圖上，如果沒有立即看到自己，請重新整理地圖。

🚀 Creating identity for AstroAyo...

🎨 Generating your portrait...

✓ Portrait generated!

🗺️ Creating map icon...

✓ Icon generated!

☁️ Uploading to mission database...

✓ Avatar uploaded!

📍 Registering with rescue network...

✓ Registration complete!

✅ IDENTITY CONFIRMED!

Explorer: AstroAyo

Location: (47, 23) — unconfirmed

🗺️ You're now on the map!

<https://waybackhome.dev/e/devfest-nyc-26>

NEXT: Proceed to Level 1 to pinpoint your exact location!

發生了什麼事？

`create_identity.py` 指令碼 (已提供給您) 執行了下列步驟：

1. 呼叫程式碼：從 `generator.py` 執行 `generate_explorer_avatar()`
2. 上傳的圖片：已將直向圖片和圖示傳送至 Mission Control API
3. 註冊身分：更新參與者記錄，加入虛擬化身網址
4. 確認成功：你的虛擬化身現已顯示在世界地圖上！

您編寫了圖像生成邏輯。指令碼會代表您處理網路和註冊事宜。

查看生成的虛擬化身

在本機查看生成的圖像：


```
# List the generated files
ls -la outputs/

# Download to view on your local machine
cloudshell download outputs/portrait.png
cloudshell download outputs/icon.png
```

或者，您也可以前往 `outputs/` 資料夾並點選圖片檔，直接在 Cloud Shell 編輯器中查看。

生成的肖像和圖示如下圖所示：





在地圖上查看自己的位置！

開啟輸出內容中顯示的世界地圖網址：

`https://waybackhome.dev/e/{your-event-code}`

例如：`https://waybackhome.dev/e/devfest-nyc-26`

在地圖上尋找你的顯示圖片！您會看到：

- 標示您在地球表面位置的圖示
- 圖示下方的標籤會顯示檔案總管名稱
- 標記周圍緩慢閃爍的微弱信號



按一下標記，即可在詳細資料卡片中查看完整肖像。

為什麼信標亮度較暗？

你目前的位置「未確認」：你位於座標 (47, 23) 區域的某處，但救援網路尚不知道你的*exact*位置。你迫降了，還記得嗎？

在「第 1 級：精確定位」中，您將使用多代理系統分析周遭環境，確認精確座標。確認後，信號燈就會亮起，方便搜救隊找到你！

不滿意虛擬化身嗎？重新生成！

如要使用其他虛擬化身，可以重新生成：

```
# Option 1: Change your appearance settings first
uv run python customize.py

# Option 2: Just regenerate with current settings
uv run python create_identity.py
```

每次執行 `create_identity.py` 時，系統都會生成**全新的**虛擬化身 (新的即時通訊工作階段 = 新的角色)，並更新註冊資訊。歡迎反覆調整，直到滿意為止！

為什麼每次執行都會建立不同角色

再次執行 `create_identity.py` 時，系統會建立新的即時通訊工作階段。由於沒有先前的脈絡，Gemini 會生成全新的角色。

這其實是我們所樂見的結果，因為這樣一來，如果對第一個結果不滿意，就能重新生成。一致性技術 (多輪對話) 可確保單次執行時，肖像和圖示一致，但每次執行都會產生獨一無二的角色。

疑難排解

常見問題

「無法生成肖像 - 回應中沒有圖片」

- 模型可能因內容政策而拒絕生成圖片。請再試一次。
- 確認外觀描述不含限制內容。
- 請確認你使用的模型支援圖片生成功能 (`gemini-2.5-flash-image`)。

「ModuleNotFoundError: No module named 'google.genai」

- 從 `level_0` 目錄重新執行 `uv sync`，確保已安裝依附元件
- 使用 `uv run python generator.py` 在正確的環境中自動執行

「權限遭拒」或驗證錯誤

- 確認已設定專案：`export GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project)`
- 確認已啟用 Vertex AI API：`gcloud services enable aiplatform.googleapis.com`

「上傳失敗」或 API 錯誤

- 在 Cloud Shell 中檢查網際網路連線。
- 確認活動是否仍有效 (活動會在研討會結束後失效)。
- 請向研討會講師確認 Mission Control API 是否正常運作。

地圖上未顯示虛擬人偶

- 請等待 5 到 10 秒，地圖會定期輪詢更新。
- 強制重新整理頁面 (Ctrl+Shift+R 鍵或 Cmd+Shift+R 鍵)。
- 確認終端機輸出內容顯示註冊程序已完成，且未發生錯誤。

您剛完成的項目

- ✓ 生成專屬的探索者肖像
- ✓ 建立一致的地圖圖示
- ✓ 將資產上傳至任務控制中心
- ✓ 向救援網路註冊身分
- ✓ 出現在即時世界地圖上！

恭喜，探險家！你現在已加入救援網路。信標會亮起 (但亮度較暗)，其他倖存者可以在地圖上看到你。

7. 結論

建構內容

您在短短 10 分鐘內，就使用多輪 AI 圖像生成功能，建立完整的虛擬化身生成系統：

Your Creation

 Input

- Explorer name
- Suit color
- Appearance description

 Output

- Stylized portrait
- Consistent map icon
- Live map presence

 Techniques You Used

- Multi-turn chat sessions for character consistency
- Structured prompt engineering with style constraints
- Response parsing to extract generated images
- API integration for cloud registration

重點回顧

課程	重要性
對話工作階段會保留脈絡	在多張圖片中生成一致的角色，避免變異漂移
提示結構很重要	明確的限制 (白色背景、特定風格、構圖) 可產生可預測且可直接用於製作的結果
多輪對話優於獨立呼叫	對話中的每則訊息都會以前後文為基礎，因此可反覆修正並保持一致性
回應模式可控制輸出內容	生成圖片時必須設定 ["TEXT", "IMAGE"]，否則只會產生文字

費用摘要

在本層級中，您使用 Gemini 2.5 Flash (Nano Banana) 生成了 2 張圖片：

項目	符記數	費用
直向 (1024×1024)	約 1,290 個輸出詞元	~\$0.039
圖示 (1024×1024)	約 1,290 個輸出詞元	~\$0.039
總計	約 2,580 個權杖	~\$0.08 美元

替代模型：Nano Banana Pro

如要生成更高品質的輸出內容，建議改用 `gemini-3-pro-image-preview` (Nano Banana Pro)。這項功能可生成更精細、更逼真的圖片，但肖像 + 圖示的費用約為 **\$0.27** (使用 1K 解析度)。

如果研討會參與者眾多，Nano Banana (Gemini 2.5 Flash) 的品質與成本比非常出色。

旅程仍在繼續

系統已確認你的探索者身分，但發生問題：**無法確認你的位置。**

查看世界地圖，您的信標會呈現微弱的亮度，並緩慢閃爍。救援網路知道你存在，但不知道你的確切位置。您可能位於星球表面廣大區域的任何位置。

在「第 1 級：找出您的位置」中，您將：

- 使用 Google 的 Agent Development Kit (ADK) 建構**多代理系統**
- 建立 **MCP 伺服器**，用於地質、植物和天文分析
- 處理**多模態輸入內容** (當機網站的圖片)
- 使用 AI 生成確認位置的**地形圖**
- **啟動信標**，讓搜救隊伍找到你！

第 1 級會介紹更進階的概念：多代理程式協調、Model Context Protocol (MCP) 和正式環境部署模式。您將從使用單一模型功能，進展到建構完整的代理系統。

清除 (選用)

如要清理本機檔案 (從 `level_0` 目錄執行)：

```
rm -rf outputs/
deactivate # Exit virtual environment
```

你仍會註冊救援網路，即使刪除本機檔案，地圖上仍會顯示你的位置。圖片會儲存在雲端，而非本機。

資源

- [Gemini 圖像生成說明文件](#)
- [圖片提示工程指南](#)
- [Nano Banana 公告網誌](#)

- [Way Back Home](#) 存放區
- [第 1 級：精確找出位置](#)

信標正在等待確認。我們第 1 級見，探索者。🚀

8. 額外步驟：以相片為基礎的虛擬化身 (選填)

選用章節

這個加分單元會說明如何使用真實相片做為虛擬人像的基礎，並運用 Gemini 的圖像轉圖像功能，將相片轉換成風格化的太空探險家美學。如果對文字生成的虛擬化身感到滿意，請略過這個步驟！

使用相片製作虛擬化身的時機

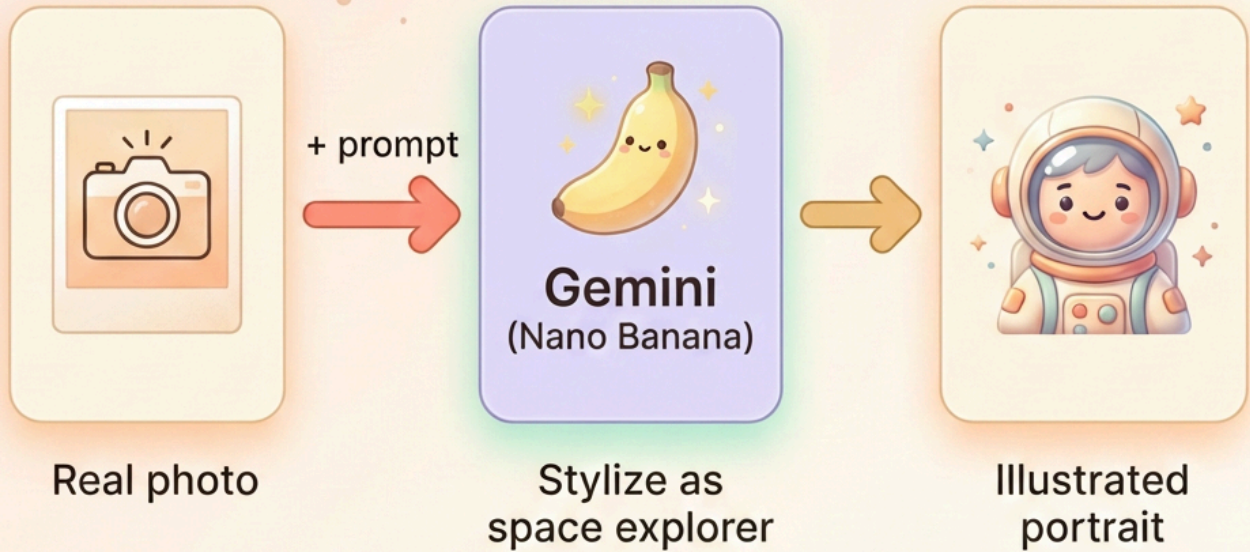
在下列情況下，您可能會偏好這種做法：

- 希望虛擬人偶與你本人相似
- 您想為特定相片套用風格
- 您想探索 Gemini 的圖片轉換功能

運作方式

你不用只提供文字描述，而是提供相片，並問問 Gemini 轉換相片，同時保留人物的樣貌：

Photo-to-Avatar Flow



上傳相片

首先，將相片上傳至 Cloud Shell：

1. 在 Cloud Shell 中，點選終端機工具列中的三點選單 (⋮)
2. 選取「上傳」
3. 選擇清晰的證件照 (JPEG 或 PNG)
4. 請記下上傳的路徑 (例如 `/home/your-username/my_photo.jpg`)

修改生成器

如要使用相片，請在 `generator.py` 中修改肖像生成步驟。將只有文字的提示換成包含相片的多模態提示：

```

# MODULE_5_STEP_2_GENERATE_PORTRAIT (Photo-based version)

# Load your photo
photo_path = "/home/your-username/my_photo.jpg" # Update this path!
user_photo = Image.open(photo_path)

# Convert photo to bytes for the API
photo_buffer = io.BytesIO()
user_photo.save(photo_buffer, format="JPEG")
photo_bytes = photo_buffer.getvalue()

portrait_prompt = f"""Transform this person into a stylized space explorer portrait.

PRESERVE from the original photo:
- The person's facial features, face shape, and likeness
- Their general expression and personality
- Any distinctive features (glasses, facial hair, etc.)

TRANSFORM with this style:
- Digital illustration style, clean lines, vibrant saturated colors
- Add a futuristic space suit with the name "{USERNAME}" on a shoulder patch
- Suit color: {SUIT_COLOR}
- Background: Pure solid white (#FFFFFF) - no gradients or elements
- Frame: Head and shoulders, 3/4 view
- Lighting: Soft diffused studio lighting
- Art style: Modern animated movie character (Pixar/Dreamworks aesthetic)

The result should be clearly recognizable as THIS specific person, but illustrated as a
heroic space explorer."""

print("🌌 Transforming your photo into an explorer portrait...")

# Send both the prompt AND the image
portrait_response = chat.send_message([
    portrait_prompt,
    types.Part.from_bytes(data=photo_bytes, mime_type="image/jpeg")
])

# Rest of the extraction code stays the same...

```

多輪對話一致性仍可運作

以相片為基礎的生成方式也適用於多輪對話：

- **第 1 輪**：相片 + 提示 → 經過風格化的肖像 (你的相似度，插畫風格)
- **第 2 輪**：「Create icon of this SAME character」(建立這個相同角色的圖示) → 一致的圖示

由於即時通訊工作階段會記住建立的風格化版本 (而非原始相片)，因此圖示會與轉換後的肖像完全相符。

隱私權注意事項

隱私權注意事項

上傳相片時：

- 相片會傳送至 Vertex AI (Google Cloud) 進行處理
- 風格化結果 (而非原始相片) 會儲存在救援網路中
- 只使用您願意交由 AI 服務處理的相片
- 文字生成功能 (不含相片) 一律可用，是隱私權保護的替代方案

在 Vertex AI Studio 中試用

您也可以在編寫程式碼前，先在 Google Cloud 控制台以互動方式試用相片轉換功能：

1. 前往 [Vertex AI Studio](#)
2. 選取具備圖片功能的 Gemini 模型
3. 使用附件按鈕上傳相片
4. 輸入變形提示詞
5. 反覆調整樣式，直到滿意為止

這種互動式方法非常適合用來實驗提示，並在確定程式碼前即時查看結果！

您學到的內容 (加分題)

- ✓ 如何使用 Gemini 的「圖片到圖片」轉換功能
 - ✓ 在單一訊息中傳送多模態內容 (文字 + 圖片)
 - ✓ 在套用藝術風格轉換時保留相似度
 - ✓ 以相片為基礎生成 AI 內容時的隱私權注意事項
-